

ATTENSE — Final Technical Review

Branch: refactor/restructure-and-pipeline-integration

Date: June 2026

Prepared by: Architecture Review

1. Bugs to Address

Critical — System Cannot Function Correctly Without These

BUG-01 • Ingest Seam (Scoring Pipeline Disconnect)

Red-team events are submitted via `(POST /api/incidents/events)` → `(controller.process_event())` and live only in the controller's in-memory dict. The scoring pipeline (`(run_bridge)`) reads `(mapped_events.jsonl)`, which is only written by `(signal-store)`. These two streams never meet.

Result: TTD calculations have no `(malicious_action_executed)` start time because no red-team event ever reaches the file the scoring engine reads. TTD will always be zero or fall back to the first `(alert_raised)` for both start and detection time — a metric that is structurally meaningless.

Fix: Unify both streams. Either write red-team HTTP events to `(mapped_events.jsonl)` as well, or port the scoring engine to read from the controller's in-memory state directly.

BUG-02 • `(generate_report())` is a Stub

`(ATTENSE_app/reports/report.py)` returns only `({ttd, ttc, outcome, incident_id, scenario_id, status, start_time, detection_time, containment_time, end_time})`. No scoring formula is ever called. `(live_thresholds.py)` was built with canonical CVSS values and TTC decay formulas but nothing imports or calls it. The penalty rules (R01-R05), TTC linear decay score, bonus calculation, and verdict band `(excellent/acceptable/needs_review/failed)` have never been wired into a live report.

Fix: Import `(live_thresholds.compute_live_thresholds(scenario_id))` inside `(generate_report())` and feed the result into the scoring formula already proven correct in `(scoring_engine.py)`.

BUG-03 • No AI Report Generated on Room Close

`(spin_down_room())` sets status to `(closed)` and removes the container. Nothing else happens. The `(Room → generates → AI_REPORT)` relationship exists in the ERD and the relational model has an `(AI_REPORT)` table, but `(spin_down_room())` never calls `(generate_report())`, never creates a report record, and never creates `(USER_REPORT)` access rows for participants.

Fix: At the end of `spin_down_room()`, call `generate_report()` for every incident in `room.incident_ids`, aggregate results, write one `AI_REPORT` record, and create `USER_REPORT` rows (`view_type=full` for blue team, `score_only` for red team) from `room.blue_team_members` and `room.red_team_members`.

BUG-04 • Red Team Room Join — Missing Entirely

Red team users can authenticate and run attack modules, but there is no API endpoint to join a room. `room.red_team_members` is always an empty list. The `initiates` relationship (User → Incident) and the `performs` relationship (User → Event) both depend on knowing which room a red team user is operating in at any given moment. Without a join endpoint, `actor_id` on events cannot be validated against a room's authorized participants.

Fix: `POST /api/rooms/{room_id}/join` — validates the user's session token, verifies their role is `expert` or `intermediate`, appends them to `room.red_team_members`, and returns the room's `incident_id` so their attack module can tag events correctly.

BUG-05 • `signal-store` INCIDENT_ID and Room Binding Are Disconnected

The room injection fix (passing the room's `incident_id` as `INCIDENT_ID` env var to the blueteam container) only affects the blueteam service. `signal-store` resolves its own `incident_id` via `_resolve_incident_id()` — which uses `wazuh-{alert.id}` for real Wazuh alerts, completely independently of any room. In a live exercise, Wazuh-generated `alert_raised` events will carry a different `incident_id` than the `malicious_action_executed` events from the red team. The state machine will treat these as separate incidents and TTD will never be calculated correctly.

Fix: Configure `signal-store`'s `INCIDENT_ID` env var per-room at `spin_up_blueteam()` time, using the same value injected into the blueteam container. This requires adding `signal-store` to the per-room configuration step.

Important — Correctness Gaps

BUG-06 • Public Exposure Hardening Unconfirmed

The prompt to move `VIRUSTOTAL_API_KEY`, `ABUSEIPDB_API_KEY`, `HIVE_API_KEY`, and `WEBHOOK_SECRET` out of `docker-compose.yml` into `.env`, and to strip host-port mappings from `target-agent` (8081), `thehive` (9000), `red-team-frontend` (3000), and `signal-store` (8005), was given but never confirmed applied. `target-agent` is an intentionally vulnerable Flask app — it must not be reachable from any public network under any circumstances.

Fix: Verify `docker-compose.yml` state. Run `docker compose config | grep ports` and confirm only `attense-app:8020` is bound to the host.

BUG-07 • `actor_id` Attribution for Wazuh Events

`mapper.py` hardcodes `actor_id = "wazuh"` on all `alert_raised` events (the comment itself says "Phase 3 correlation engine will use `source_ip` to attribute the attack — not yet built"). This means the `performs` relationship (User → Event) is unresolvable for system-generated events. The `source_ip` in metadata is the attacker's IP, not a `User.id`, so the correlation that would say "this alert was caused by `red_team_user_X`" is missing.

Status: Known architectural gap. Acceptable for thesis scope — document explicitly.

BUG-08 • Zero Day Entity Attributes Not Confirmed

The Zero Day agent is implemented (teammate's work, main branch) but the entity attributes in the ERD (`zd_id`, `detected_at`, `technique`, `signature`) were not verified against the actual implementation because the main branch could not be indexed during this review. The ERD attributes are based on the proposal's stated requirements.

Fix: Open the zero-day agent implementation file, compare its stored fields against the ERD entity, and update any mismatches before finalizing the schema with the doctor.

Minor — Technical Debt

BUG-09 • No CISO List-All-Companies Endpoint

A CISO can confirm a company by ID but has no endpoint to see all active companies, not just pending ones. `GET /api/company/pending` exists but `GET /api/company/` (all) does not.

BUG-10 • `port` Column in Room Schema

`Room.port` (8110–8130) is a runtime infrastructure detail tracked in the JSON store. It should not be in the production DB schema — ports are ephemeral and meaningless after a room closes. Recommend omitting from Postgres migration and using the service registry instead.

BUG-11 • `score` Stored on User (Relational Model)

The doctor's ERD shows `score` as a dashed (derived) attribute on User. The relational model should not have a `score` column on the User table — scores belong on `AI_REPORT` per exercise. A user's lifetime score is a view-layer computation, not a persisted field.

2. Next Implementation Plan

Priority 1 — Without These, the Core Loop Does Not Work

#	Task	Estimated Effort
P1-1	Fix BUG-01: Unify red-team and Wazuh event streams for scoring	Medium
P1-2	Fix BUG-02: Wire <code>live_thresholds</code> into <code>generate_report()</code> with full penalty/verdict	Medium
P1-3	Fix BUG-03: Trigger AI report + USER_REPORT rows on <code>spin_down_room()</code>	Small
P1-4	Fix BUG-04: <code>POST /api/rooms/{id}/join</code> for red team users	Small
P1-5	Fix BUG-05: Configure signal-store INCIDENT_ID per room at spin-up	Small

Priority 2 — Required for a Complete Demo

#	Task	Estimated Effort
P2-1	Database migration (JSON → Postgres) using the confirmed ERD schema	Large
P2-2	Blue team portal UI — thin login page that authenticates analysts and redirects to TheHive with their <code>hive_key</code>	Small
P2-3	Confirm Zero Day agent attributes match ERD, update schema if needed	Small
P2-4	Fix BUG-06: Public exposure hardening (confirm or apply)	Small
P2-5	Report view endpoint — <code>GET /api/rooms/{id}/report</code> returning full or score-only based on caller's role	Small

Priority 3 — Quality and Polish

#	Task	Estimated Effort
P3-1	Fix BUG-09: <code>GET /api/company/</code> (list all companies for CISO)	Trivial
P3-2	Remove <code>port</code> from Room DB schema, use service registry	Small
P3-3	Fix BUG-11: Remove <code>score</code> from User table	Trivial
P3-4	End-to-end integration test: full exercise loop (create room → red team attacks → blue team responds → room closes → verify AI report generated)	Medium

3. Hosting Strategy

The Short Answer: Not an EXE

An executable is not feasible for this system. The stack runs 20+ Docker containers including Wazuh (kernel-level SIEM that requires Linux), TheHive (Java + Cassandra + Elasticsearch, minimum 8GB RAM together), Redis, and multiple Python services. None of these can be bundled into a Windows executable. Docker is a hard dependency.

Recommended Approach: Three Tiers

Tier 1 — Local demo (today)

Keep the current Docker Compose stack as-is. Anyone with Docker Desktop installed runs:

```
docker compose up -d
```

That is your "installer." For temporary public access during demos or doctor reviews, use Cloudflare Tunnel — it takes 30 seconds and exposes only `attense-app:8020`:

```
cloudflared tunnel --url http://localhost:8020
```

Free, stable URL, automatic TLS, no firewall changes, no port forwarding. The tunnel closes when you stop the command. This is appropriate for the thesis defense and short-term testing.

Tier 2 — Cloud VM (after defense, for continued access)

For longer-term access where others need to test independently, a cloud VM is the correct answer. The `(docker-compose.yml)` deploys identically on any Linux VM.

Minimum spec for the full stack: **4 vCPU, 16GB RAM** (TheHive + Cassandra + Elasticsearch alone consume ~8GB). Recommended providers:

Provider	Instance	Monthly Cost
AWS EC2	t3.xlarge	~\$120
DigitalOcean	4vCPU/16GB Droplet	~\$96
Hetzner Cloud	CPX31	~\$18

Hetzner is the most cost-effective for a thesis project. Setup is identical to local — clone the repo, run `(docker compose up -d)`, point a Cloudflare tunnel or the VM's public IP at port 8020. Budget for 2-3 months of hosting during the testing/grading period.

Tier 3 — What NOT to do

Do not attempt to deploy to AWS ECS, Kubernetes, or any container orchestration platform for a thesis project. The complexity of adapting the docker-compose networking (especially Wazuh's custom network and TheHive's multi-service dependencies) to a managed container platform would consume more time than the remaining implementation work.

Do not expose the full stack publicly. Only `(attense-app:8020)` should ever be internet-facing. `(target-agent)` (intentionally vulnerable), `(thehive)` (admin credentials exposed), and `(wazuh-manager)` (management ports) must never have public IPs or open ports.

4. Summary

Category	Count
Critical bugs	5

Category	Count
Important gaps	3
Minor / tech debt	3
P1 tasks (required for working demo)	5
P2 tasks (complete demo)	5
P3 tasks (polish)	4
Hosting recommendation	Cloudflare Tunnel (demo) · Hetzner VM (long-term)

The core logic — state machine, event pipeline, scoring engine against test fixtures, room orchestration, TheHive isolation, auth, red team tooling — is all built and verified. What remains is connecting the last three joints: red-team events reaching the scoring engine (BUG-01), the scoring engine producing a real output (BUG-02), and that output being persisted and served when a room closes (BUG-03). Everything else builds on top of those three fixes.